

מבחן מעשי: ניתוח קובצי BLF (dGPS + FrameID)

זמן מוקצב: יום עבודה אחד (להגשה עד לשעה שסוכמה מראש).

רקע

בצוות אנו מקליטים בזמן מסלול ניסוי (Test Track) נתוני dGPS של שני אובייקטים:

1. VUT (Vehicle Under Test) רכב הניסוי.

2. Target / GVT (Guided Soft Target) המטרה.

הנתונים הללו, יחד עם ה-FrameID ממערכת ה-EyeQ, נרשמים יחדיו לתוך קובץ לוג מסוג BLF

המשימה שלך: לחלץ מהלוג את הנתונים המפוענחים, לייצא אותם לקובץ CSV ממוזג, ולבנות כלי ויזואליזציה יעיל (גרפים, דשבורד או דוח אינטראקטיבי) לצורך ניתוח הריצה.

מה זה FrameID ?

FrameID הוא מונה פריימים פנימי של מערכת מחשב התפיסה של Mobileye. כל פריים חדש שמסגרת התפיסה (perception) מעבדת מקבל מספר עולה. זהו למעשה ציר הזמן הרשמי של מערכת התפיסה.

מפרט הודעת ה-FrameID:

פרמטר	ערך
קובץ DBC	ISR_mqttToCan_dbc_251210.dbc
CAN ID	(0x400 עשרוני)1024 :
שם הודעה	mqtt_to_can_bridge_1_2_3_bridge_1_2_3_Bridge
שם סיגנל	frameID
תדר שידור	(100 Hz ~הודעה בכל כ10 ms -

משמעות הפרמטר בפועל:

- סנכרון: ה-FrameID מאפשר לקשר באופן חד-ערכי בין נתוני התפיסה לבין ה-Ground-Truth הפיזיקלי המגיע מה-DGPS שניהם חיים על אותו ציר זמן של ה-BLF
- מונטוניות: בנסיעה תקינה ערך ה-FrameID חייב לעלות ברצף (בדרך כלל בקפיצות של \$+1 בכל הודעה). קפיצות גדולות או חורים מעלים חשד מיידי לבעיית סנכרון. (sync drop)
- זהירות - FrameID אינו חותמת זמן (UTC) ואינו קואורדינטה גיאוגרפית. הוא אינדקס רץ ופנימי בלבד.

משימות המבחן

חלק א' – חילוץ ומיזוג נתונים (חובה)

1. קריאה ופענוח: קרא את קובץ ה-BLF והשתמש בשני קובצי ה-DBC שסופקו כדי לפענח את הסיגנלים.

2. ייצוא קובץ: CSV הפק קובץ בשם `dgps_frameid_export.csv` שיכיל לפחות את העמודות הבאות:

שם עמודה ב-CVS-	תיאור הסיגנל
<code>timestamp_sec</code>	זמן הודעת ה-CAN (בשניות) ציין ב-README האם יחסי או מוחלט)
<code>frameID</code>	ה Frame ID מתוך הודעת ה CAN ID (MQTT2CAN-0x400)
<code>VUT_Velocity</code>	מהירות אופקית של ה VUT ביחידות $[m/s]$
<code>Target2_Velocity</code>	מהירות אופקית של ה Target ביחידות $[m/s]$
<code>Target2_FW_Distance</code>	המרחק הקדמי הפרונטלי בין שני האובייקטים ביחידות $[m]$
<code>RangeTimeToCollisionForward</code>	זמן להתנגשות צפוי קדימה (TTC) בשניות $[s]$
<code>VUT_Lat_Meter</code>	מיקום מקומי של ה VUT על ציר ה Y ביחידות $[m]$
<code>VUT_Lng_Meter</code>	מיקום מקומי של ה VUT על ציר ה X ביחידות $[m]$
<code>TargetPosLocalX</code>	מיקום מקומי של ה Target על ציר ה X ביחידות $[m]$
<code>TargetPosLocalY</code>	מיקום מקומי של ה Target על ציר ה Y ביחידות $[m]$

3. מיזוג על ציר הזמן: (Time Alignment) מאחר שסיגנלי CAN שונים מגיעים בקצבים וב IDs שונים, עליך לבצע אינטגרציה ביניהם.

○ תעד ב-README באיזו גישה בחרת): למשל: דגימה מחדש ל 10 ms -שימוש ב-`forward-fill` או מיזוג אסינכרוני באמצעות `pandas.merge_asof`.

4. DBC.

חלק ב' – ויזואליזציה (חובה)

הפק לפחות 4 גרפים (כקובצי PNG, קובץ HTML האפשרים לנתח את מהלך הנסיעה:

- גרף מהירויות: פרופיל המהירות של ה VUT-מול ה Target-לאורך זמן.
- גרף מרחק קדמי: השתנות המרחק Target2_FW_Distance לאורך זמן.
- גרף TTC קדמי: השתנות ה RangeTimeToCollisionForward לאורך זמן.
- גרף מונוטוניות: FrameID ה frameID-לאורך זמן (כדי לוודא שהערך עולה ברצף ללא קפיצות או נפילות).

הערה: שימוש בכלים אינטראקטיביים כמו Plotly או Dash המאפשרים תקריב (Zoom) על קטעים ספציפיים בנסיעה יתקבל בברכה.

חלק ג' – ניתוח וממצאים (חובה)

כתוב בקובץ ה README.md-ניתוח תמציתי של הנתונים (עד 15 שורות) וענה על השאלות הבאות:

- מהו משך ההקלטה הכולל ומהו קצב הדגימה המשוער?
- מהם טווחי המהירויות (מינימום ומקסימום) של ה VUT-ושל ה Target?
- האם ישנם קטעים שבהם שני האובייקטים נעים במקביל? אם כן, באילו שניות זה קורה?
- האם ה frameID-עולה ברצף מונוטוני? האם זיהית קפיצות, חורים או איבודי מידע?